

U2U INNOVATIVE INTERNSHIP PROGRAM

WEEK 5: API Testing Report

Project: PrepMate – AI-Powered Technical Interview and Project Viva Simulator

Group No. 100

Team Members:

1. Payal Mukesh Lokhande – Information Technology- UMIT,Mumbai.
2. Siddhi Suresh Shelar- Information Technology- UMIT,Mumbai.
3. Khushi Pankhaniya- Information Technology- UMIT,Mumbai.
4. Siddhi Parab- Information Technology- UMIT,Mumbai.
5. Manasvi Korde- Information Technology- UMIT,Mumbai.

Project Overview – RAG for Educational Use

Our project, PrepMate – AI-Powered Technical Interview & Project Viva Simulator, is built using Retrieval-Augmented Generation (RAG) to provide students with an intelligent and personalized learning experience. We developed this system to help students prepare effectively for technical interviews and project viva examinations by generating context-aware questions and responses based on uploaded study materials and project documents. Instead of relying solely on pre-trained AI knowledge, our system first retrieves relevant information from the uploaded documents and then uses it to generate accurate, meaningful, and educational responses.

Introduction

PrepMate is an AI-powered Technical Interview and Project Viva Simulator designed to help students prepare for technical interviews and academic project viva examinations through interactive, real-time conversations. The application provides two dedicated preparation modes: Technical Interview and Project Viva, allowing users to practice domain-specific questions, receive AI-generated feedback, evaluate their responses, and improve their communication and problem-solving skills.

The backend of PrepMate is developed using FastAPI and integrates Artificial Intelligence to generate relevant interview and viva questions, assess user responses, maintain session history, and manage uploaded resumes and project reports. The system also supports context-aware viva interactions by utilizing uploaded project documents, enabling more meaningful and personalized questioning.

Objective of API Testing:

The objective of this API testing activity was to validate the functionality and reliability of the backend services used by PrepMate. Testing was performed using Postman to verify that each API endpoint processed requests correctly, returned appropriate HTTP status codes, generated valid JSON responses, handled file uploads, and supported seamless communication between the frontend and backend components. The testing results documented in this report confirm the operational behavior of the application's APIs and their readiness for integration and demonstration.

Tool Used: Postman was used as the primary API testing tool.

Test Environment:

Component	Technology
Backend Framework	FastAPI
Database	SQLite
Testing Tool	Postman
Local Server	http://localhost:8000

API Testing:

Technical Chat API

Endpoint:

```
POST /api/v1/technical/chat
```

Purpose:

The Technical Chat API simulates an AI-powered technical interview by accepting the candidate's response and generating interview questions, evaluating answers, assigning scores, and providing constructive feedback while maintaining the interview session.

Request:

```
{  
  "session_id": "test_tech_002",  
  "message": "Hello, I am ready to start my technical interview simulation."  
}
```

Expected Result

- HTTP 200 OK response
- Valid JSON response
- Interview session initialized or continued successfully
- AI-generated technical question returned
- Candidate response evaluated with score and feedback
- Conversation stored successfully

Actual Result

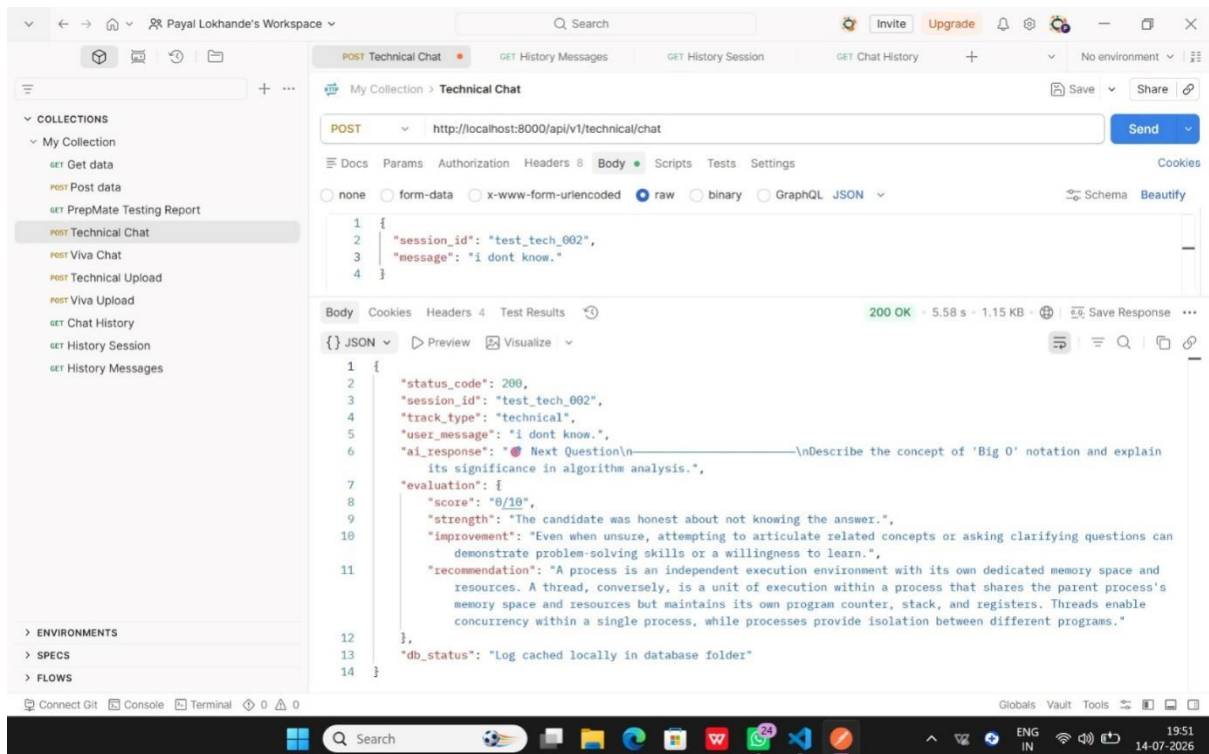
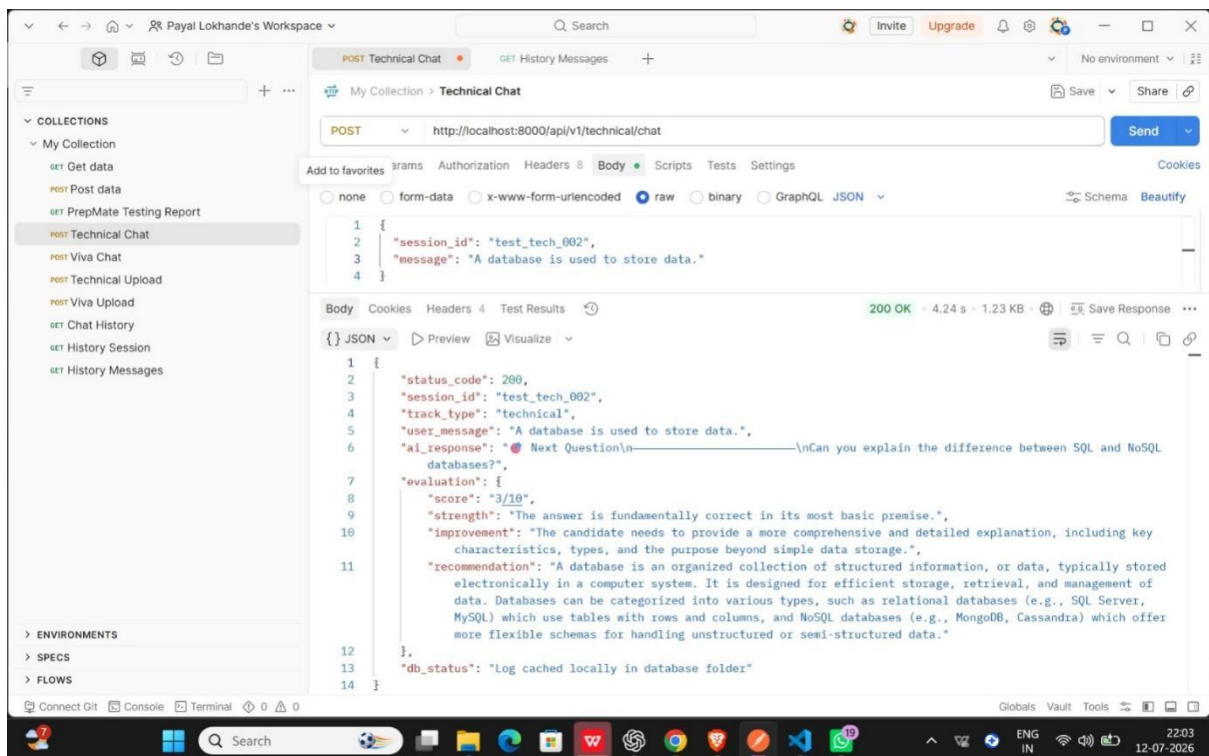
During testing, the Technical Chat API performed exactly as expected. The API returned an HTTP 200 OK response and successfully initialized the interview session. The AI generated relevant technical questions such as the difference between a Process and a Thread, evaluated different types of responses ranging from incomplete to detailed answers, assigned appropriate scores, provided constructive feedback, suggested improvements, and generated follow-up questions. All responses were returned in valid JSON format, and the interview session was successfully maintained throughout the testing process.

Status Code: 200 OK

Technical Chat API – JSON Verification:

The response received from the Technical Chat API was successfully returned in a well-structured JSON format without any syntax or parsing errors. The JSON response contained all the required fields necessary for conducting the interview, including the unique session ID, AI-generated interview questions, evaluation details, score, strengths, areas for improvement, recommendation, and the subsequent technical question. The response structure remained consistent throughout multiple test cases, regardless of whether the candidate submitted a detailed answer, a brief response, or an incorrect answer. No mandatory fields were missing, confirming that the API returns complete and standardized JSON responses suitable for frontend integration.

Screenshot:



Result: ☒ PASS

Viva Chat API

Endpoint:

```
POST /api/v1/viva/chat
```

Purpose:

The Viva Chat API simulates an AI-powered viva examination by accepting a candidate's response, generating the next viva question, evaluating the submitted answer, assigning a score, identifying strengths, providing improvement suggestions, recommending a better response, and storing the conversation in the database.

Request:

Method: POST

URL: <http://localhost:8000/api/v1/viva/chat>

Body: JSON

```
{
  "session_id": "test_viva_001",
  "message": "Good morning. My project is Smart Waste Management System Using IoT..."
}
```

Expected Result:

HTTP 200 OK with session_id, track_type, user_message, ai_response, evaluation and db_status.

Actual Result:

HTTP 200 OK was returned. Project introduction, short project descriptions, detailed technical explanations, and weak or incomplete answers were tested. The API generated relevant follow-up viva questions, evaluation scores, feedback, recommendations, and stored conversations successfully.

```
Status Code: 200 OK
```

Viva Chat API - JSON Verification: The response consistently contained status_code, session_id, track_type, user_message, ai_response, evaluation, and db_status in valid JSON format with no missing or malformed fields.

JSON Response Verification

The Viva Chat API consistently returned valid and well-structured JSON responses. All mandatory fields were present, and the generated viva questions, evaluations, and database status remained consistent across all successful requests.

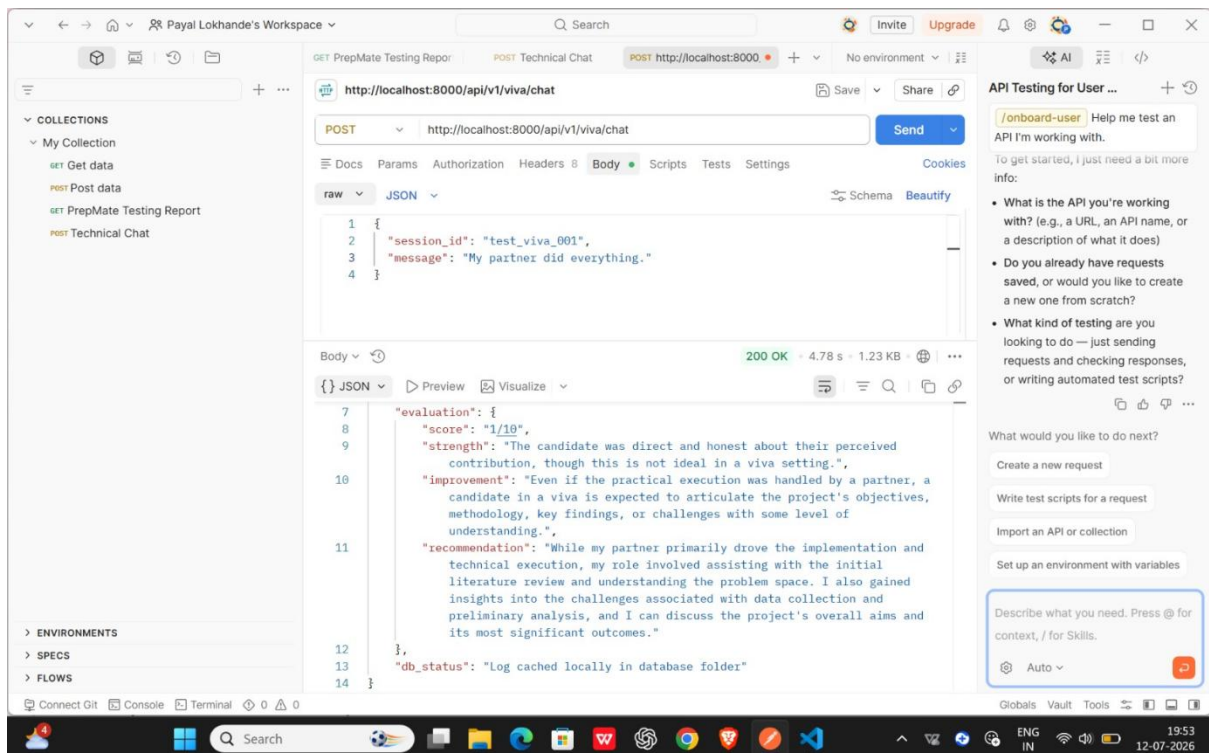
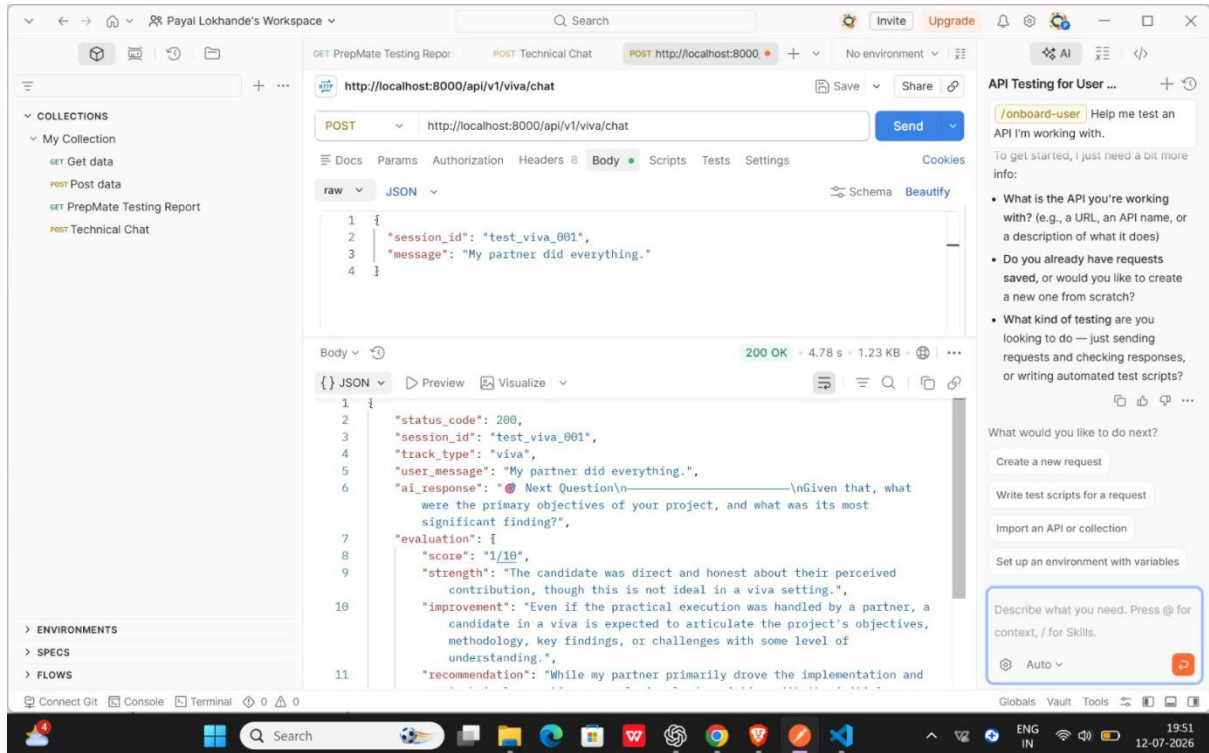
Error Handling

Only valid input scenarios were tested during this phase, including project introductions, short project descriptions, detailed technical explanations, and weak or incomplete answers. All requests returned successful HTTP 200 OK responses without server-side errors. The API consistently returned valid JSON responses, and no HTTP 400 or HTTP 500 errors were encountered.

Conclusion

The Viva Chat API successfully met all functional requirements by generating context-aware viva questions, providing detailed evaluations, returning structured JSON responses, and storing conversations successfully. The API demonstrated reliable performance and is ready for frontend integration and project demonstration.

Screenshots:



Result: ☒ PASS

Technical Upload API

Endpoint:

POST /api/v1/technical/upload

Purpose:

The Technical Upload API is responsible for uploading a candidate's resume in PDF format before starting the AI-based technical interview. After receiving the uploaded file, the backend validates the resume, creates a unique interview session, generates the first AI-based Technical interview question, and returns a structured JSON response. The purpose of testing this API was to verify that valid PDF resumes are uploaded successfully, the interview session is created correctly, and the API returns the expected response without errors.

Request:

Method: POST

URL: <http://localhost:8000/api/v1/technical/upload>

Body: form-data

Key: file

Type: File

Value: Fake-Resume.pdf

Expected Result:

When a valid PDF resume is uploaded using the correct file key, the API should return HTTP 200 OK along with a valid JSON response containing the upload status, session_id, message, and the first AI-generated interview question.

Actual Result:

The API successfully processed the uploaded PDF resume and returned HTTP 200 OK. The response contained the status, session_id, message, and first_question fields, confirming successful processing, interview session creation, and AI question generation.

Status Code: 200 OK

Technical Upload API - JSON Verification:

The response contained all required fields (status, session_id, message, and first_question) in valid JSON format. The JSON structure was consistent with the expected API specification and can be reliably used by the frontend application.

JSON Response Verification

The Technical Upload API consistently returned valid and well-structured JSON responses for every successful upload request. Each response contained all mandatory fields, including status, session_id, message, and first_question, without any missing, incorrect, or malformed values. The response format remained consistent throughout testing, ensuring seamless communication between the backend and frontend.

Error Handling :

An invalid multipart form-data request using the key Resume instead of file returned HTTP 422 unprocessable Content with a validation error. After correcting the key to file, the API returned HTTP 200 OK with the expected JSON response, confirming proper request validation.

Conclusion

The Technical Upload API successfully uploaded PDF resumes, created interview sessions, generated AI-based technical interview questions, returned structured JSON responses, and handled invalid requests appropriately. The API met all functional requirements and is ready for frontend integration and project demonstration.

Screenshots:

The screenshot shows the Postman interface with a workspace named 'Payal Lokhande's Workspace'. The 'Technical Upload' collection is selected, and a POST request is configured to 'http://localhost:8000/api/v1/technical/upload'. The 'form-data' tab is active, showing a single key-value pair: 'file' (File type) with the value 'Fake-Resume.pdf'. The 'Body' tab shows a successful response (200 OK) in JSON format:

```
{
  "status": "success",
  "session_id": "technical_1783867619",
  "message": "Resume uploaded successfully.",
  "first_question": "Tell me about yourself and briefly explain one project from your resume."
}
```

The right sidebar displays the 'API Testing for User...' section with a list of questions and a 'Describe what you need' input field.

The screenshot shows the Postman interface with the same workspace and collection. The POST request is configured to 'http://localhost:8000/api/v1/technical/upload'. The 'form-data' tab is active, showing a single key-value pair: 'Resume' (File type) with the value 'Fake-Resume.pdf'. The 'Body' tab shows an error response (422 Unprocessable Content) in JSON format:

```
{
  "detail": [
    {
      "type": "missing",
      "loc": [
        "body",
        "file"
      ],
      "msg": "Field required",
      "input": null
    }
  ]
}
```

The right sidebar displays the 'API Testing for User...' section with a list of questions and a 'Describe what you need' input field.

Result: ☒ PASS

Viva Upload API

Endpoint:

POST /api/v1/viva/upload

Purpose:

The Viva Upload API is responsible for uploading a student's project report in PDF format before starting the AI-based viva examination. After receiving the uploaded file, the backend validates the PDF, securely stores it, and prepares it for further processing so that AI-generated viva questions can be created based on the report content. The purpose of testing this API was to verify that valid PDF files are uploaded successfully, the server processes the request correctly, and an appropriate JSON response is returned.

Request:

Method: POST

URL: http://localhost:8000/api/v1/viva/upload

Body: form-data

Key: file

Type: File

Value: Sample_Project_Report.pdf

Expected Result:

When a valid PDF file is uploaded using the correct file key, the API should return HTTP 200 OK along with a valid JSON response containing the uploaded filename, size, and status, confirming that the file has been successfully uploaded and is ready for further viva processing.

Actual Result:

The API successfully processed the uploaded PDF file and returned HTTP 200 OK. The response included the uploaded file's filename, size, and status, confirming that the backend validated the file, stored it successfully, and made it available for the subsequent AI-based viva question generation process. No processing errors or unexpected behavior were observed during testing.

Status Code: 200 OK

Viva Upload API - JSON Verification:

The response contained the expected fields (filename, size, and status) in valid JSON format. No missing or malformed fields were observed. The JSON structure was consistent with the expected API response and can be reliably used by the frontend application.

JSON Response Verification:

The Viva Upload API consistently returned valid and well-structured JSON responses for every successful upload request. Each response contained the required fields, namely filename, size, and status, without any missing, incorrect, or malformed values. The response format remained consistent throughout testing, ensuring reliable communication between the backend and frontend while supporting further viva processing.

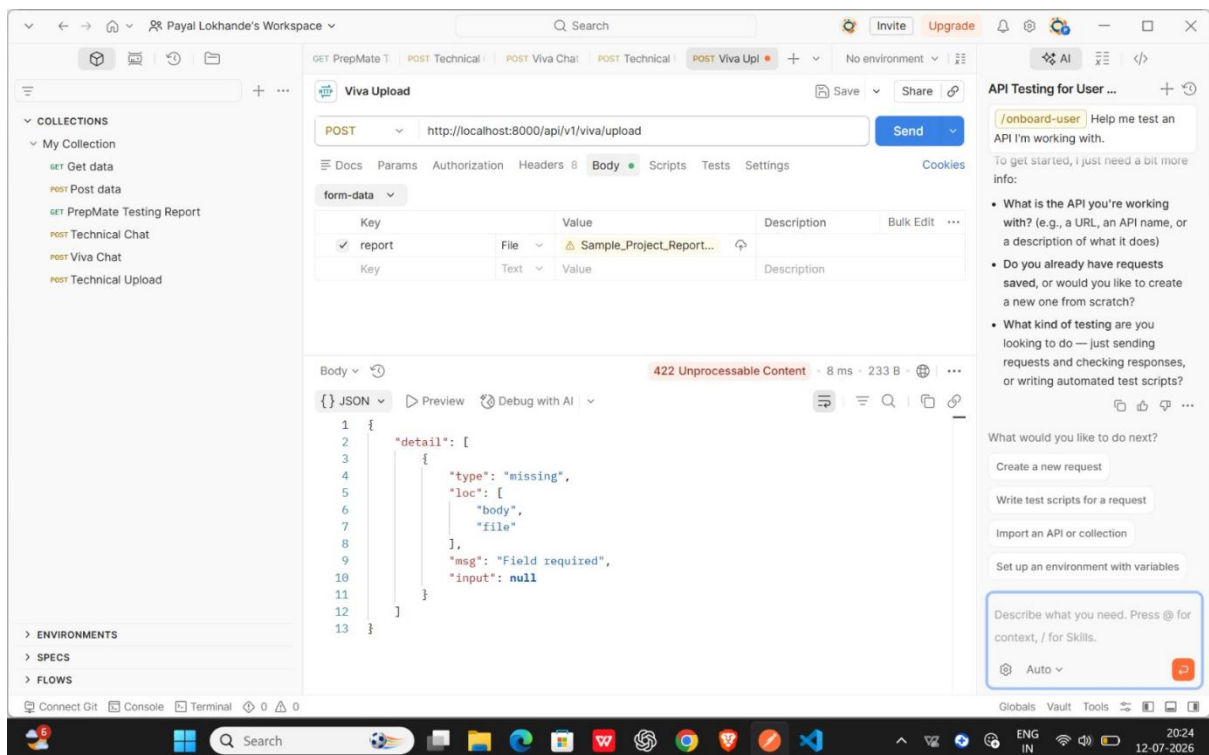
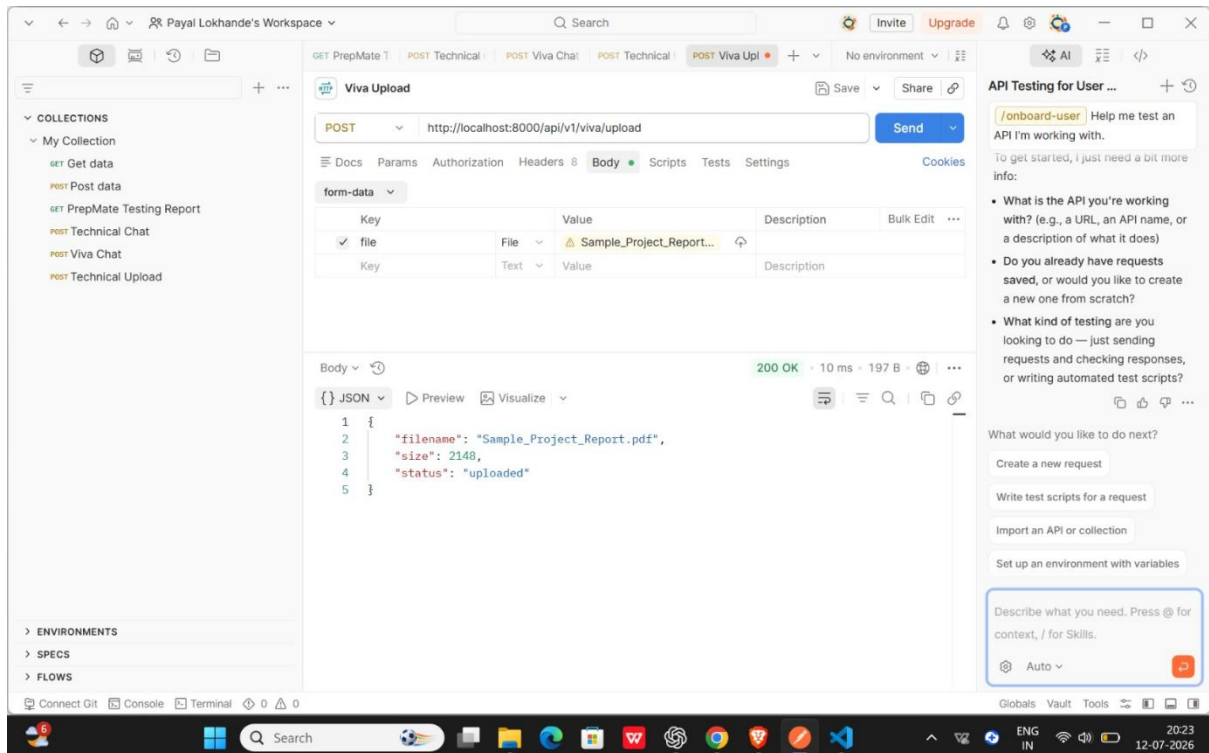
Error Handling:

Error handling was verified by sending an invalid multipart form-data request in which the file was uploaded using the incorrect key report instead of the required file. As expected, the API returned HTTP 422 unprocessable Content along with a JSON validation message indicating that the required file field was missing. After correcting the request by using the proper file key, the API successfully processed the upload and returned HTTP 200 OK with the expected JSON response. This confirms that the API correctly validates incoming requests and provides meaningful error messages for invalid input.

Conclusion:

The Viva Upload API successfully fulfilled all the expected functional requirements during testing. It uploaded valid PDF project reports without errors, returned structured and consistent JSON responses containing the required filename, size, and status fields, and correctly handled invalid requests by returning appropriate validation errors. The API demonstrated reliable performance, accurate request validation, and proper backend processing, indicating that it is ready for seamless frontend integration and AI-based viva examination workflows.

Screenshots:



Result: ☒ PASS

Chat History API

Endpoint

```
GET /api/v1/chat/history?session_id={session_id}
```

Purpose

Retrieves the stored chat history (previous messages and AI responses) for a given session, so a user can view their past conversation.

Request

```
GET http://localhost:8000/api/v1/chat/history
?session_id=test_tech_001&session_id=test_viva_001
```

session_id	test_tech_001
session_id	test_viva_001

Expected Result

- 200 OK
- Valid JSON response
- Stored chat history (messages array) returned for the session
- Each message includes session_id, user input, and AI response

Actual Result

The endpoint returned a 200 OK response, but no actual chat history data was returned. The response contained only a placeholder message:

```
{
  "message": "History endpoint ready. Connect Member 3
  retrieval function."
}
```

This indicates the endpoint route is structurally functional (reachable and responding), but the backend retrieval logic — fetching stored history from the database — has not yet been implemented. Member 3's database retrieval function is still pending integration.

Status Code

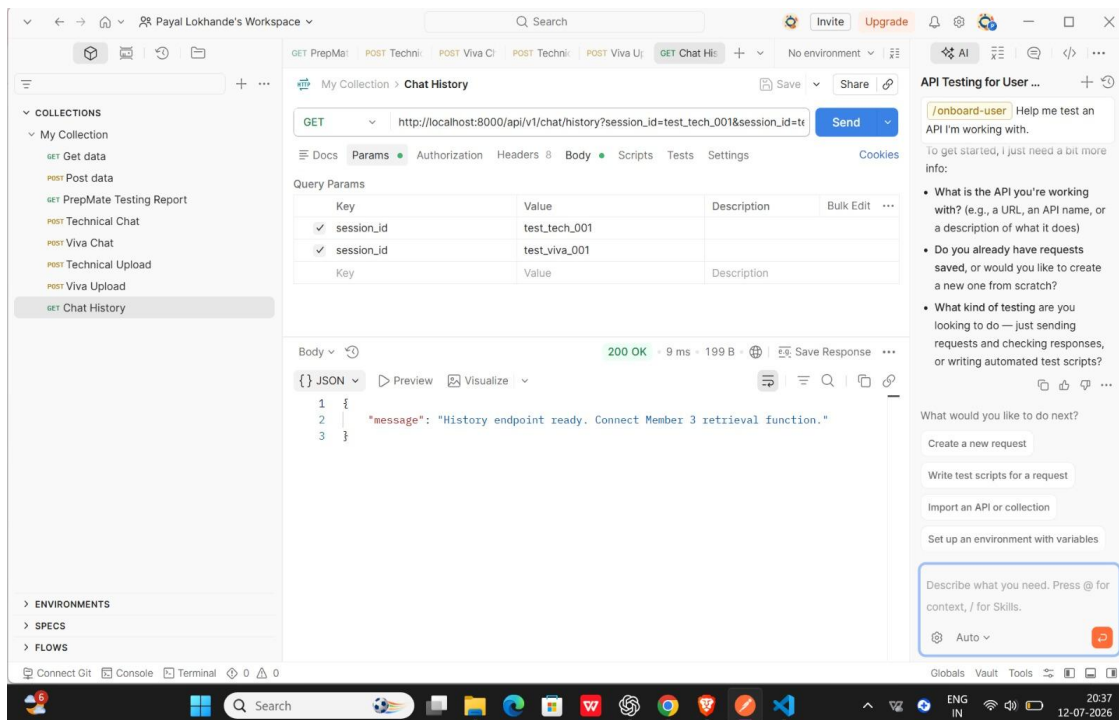
```
200 OK
```

JSON Verification

- Response received in valid JSON format
- Response time: 9 ms | Size: 199 B
- Required fields (e.g. history, messages, session_id) are not yet present in the response, since the retrieval logic is pending

Screenshot

Postman screenshot attached below — GET Chat History request, 200 OK, JSON body showing the placeholder message.



Result

⚠️ PARTIAL PASS

The endpoint is reachable and returns 200 OK, but functional testing is incomplete because Member 3's database retrieval logic is not yet connected. A full PASS will be confirmed once actual chat history data is returned in the response.

History Session API

Endpoint

```
GET /api/v1/history/sessions
```

Purpose

Retrieves the list of previous chat sessions for a user, allowing results to be filtered by track type (technical or viva) using the 'mode' query parameter.

Test Case 1: Fetch Technical Sessions

Request

```
GET http://localhost:8000/api/v1/history/sessions?mode=technical
```

Expected Result

- 200 OK
- Valid JSON array returned
- Only sessions with track_type = "technical" included
- Each session contains session_id, track_type, timestamp and title

Actual Result

The API returned a 200 OK response with a JSON array of technical sessions. Each object correctly included the session_id, track_type set to "technical", a timestamp, and a title field.

Status Code

```
200 OK
```

JSON Verification

- Response received in valid JSON format
- Required fields (session_id, track_type, timestamp, title) present in every object
- No missing fields
- track_type consistently equals "technical" across all entries

Screenshot

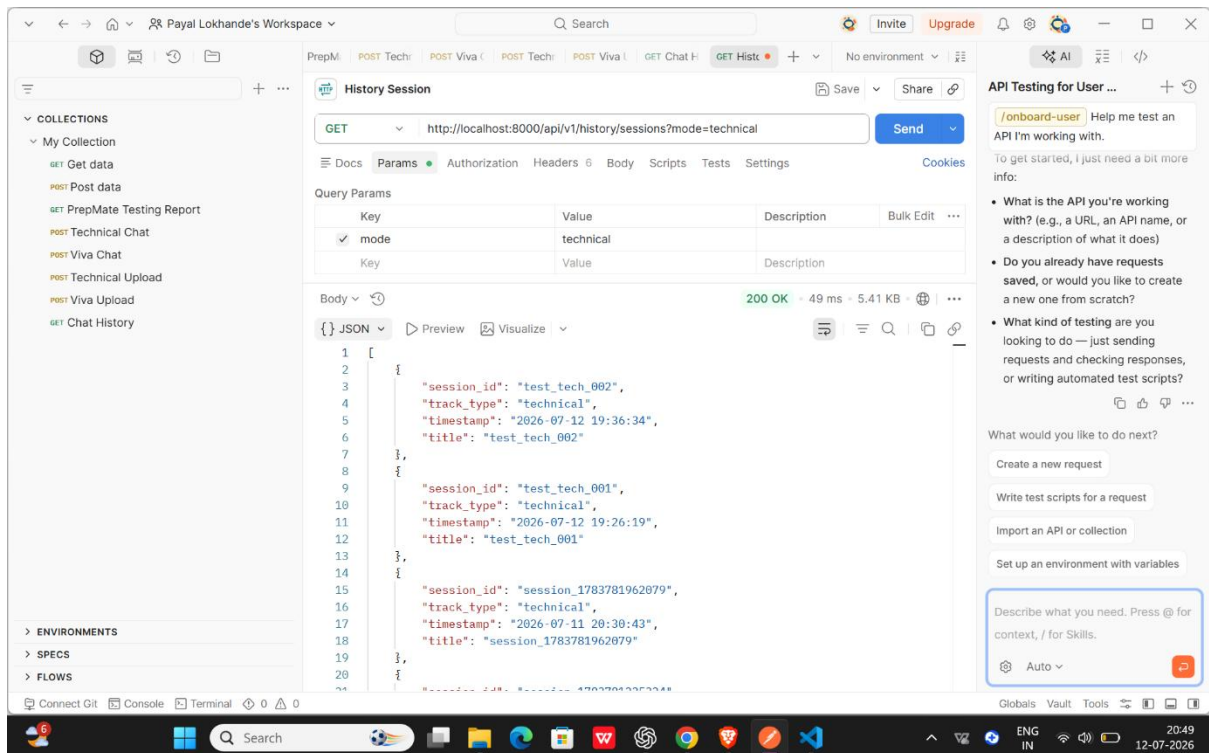


Fig 1: GET /history/sessions?mode=technical — 200 OK response

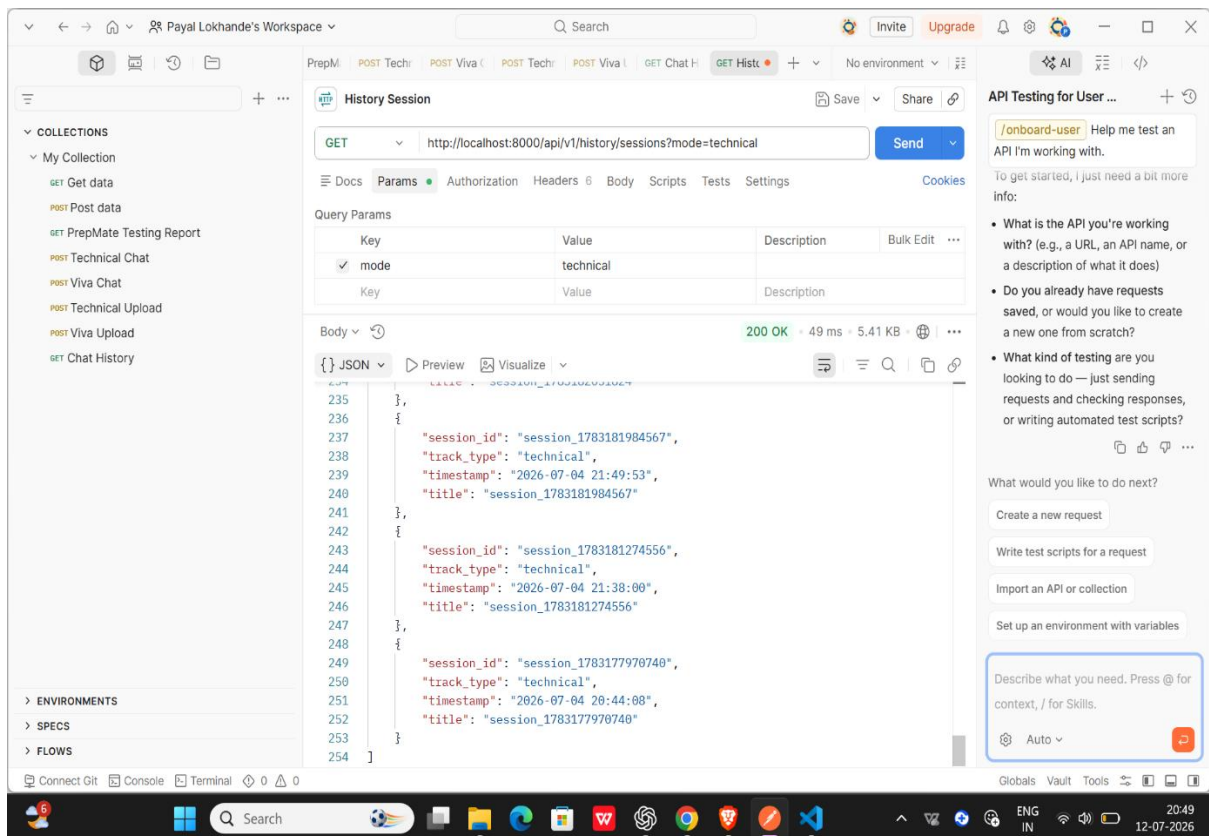


Fig 2: History session technical (scrolled view)

Result

✓ PASS

Test Case 2: Fetch Viva Sessions

Request

```
GET http://localhost:8000/api/v1/history/sessions?mode=viva
```

Expected Result

- 200 OK
- Valid JSON array returned
- Only sessions with track_type = "viva" included
- Each session contains session_id, track_type, timestamp and title

Actual Result

The API returned a 200 OK response with a JSON array of viva sessions. Each object correctly included the session_id, track_type set to "viva", a timestamp, and a title field.

Status Code

```
200 OK
```

JSON Verification

- Response received in valid JSON format
- Required fields (session_id, track_type, timestamp, title) present in every object
- No missing fields
- track_type consistently equals "viva" across all entries

Screenshot

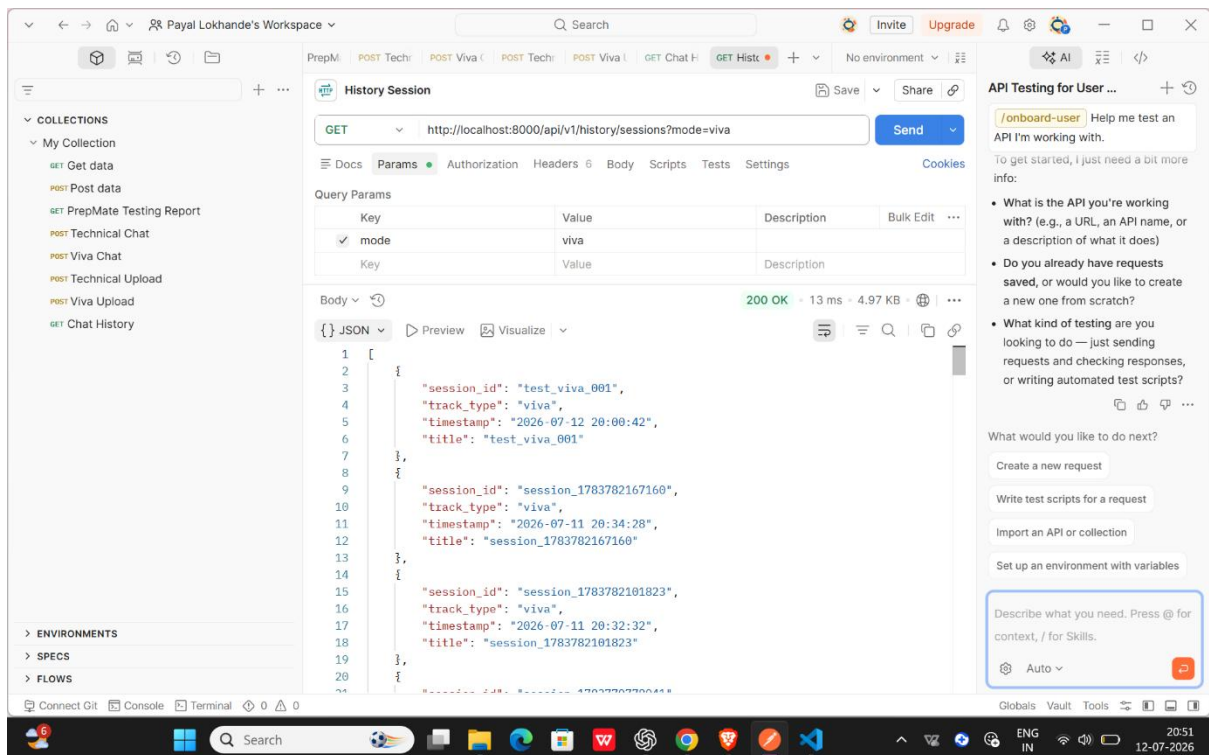


Fig 3: GET /history/sessions?mode=viva — 200 OK response

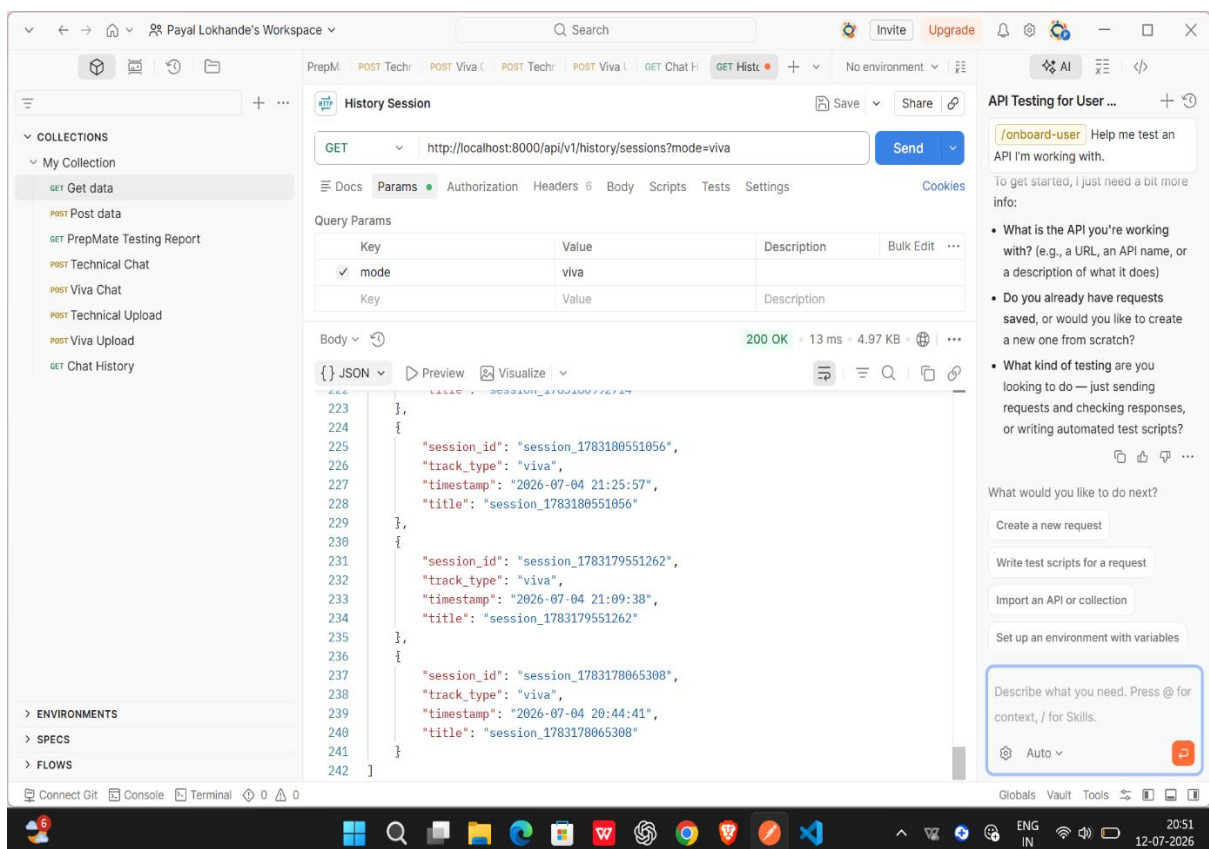


Fig 4: Viva session list (scrolled view)

Result

✓ PASS

Test Case 3: Missing 'mode' Query Parameter (Negative Test)

Request

```
GET http://localhost:8000/api/v1/history/sessions
```

Expected Result

- API should return a client error since 'mode' is a required query parameter
- Error message should clearly indicate the missing field

Actual Result

The API returned a 422 Unprocessable Content error. The response body identified 'mode' as a missing required query field, confirming that server-side validation is working as intended.

Status Code

```
422 Unprocessable Content
```

JSON Verification

- Response received in valid JSON format
- Error detail array present with type "missing", loc ["query", "mode"] and msg "Field required"
- Validation correctly rejects the request when 'mode' is absent

Screenshot

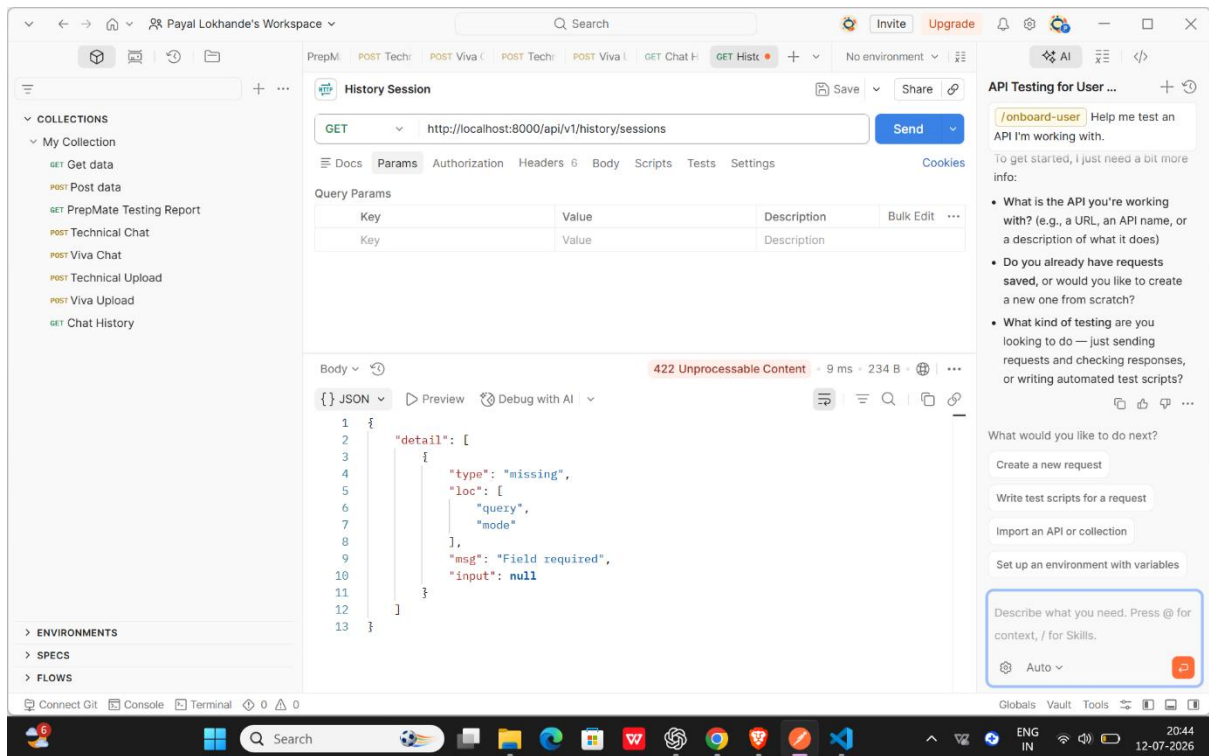


Fig 5: GET /history/sessions without 'mode' — 422 Unprocessable Content

Result



Summary

Test Case	Status Code	Result
Fetch Technical Sessions (mode=technical)	200 OK	PASS
Fetch Viva Sessions (mode=viva)	200 OK	PASS
Missing 'mode' parameter (negative test)	422 Unprocessable	PASS

Overall Result: All test cases for the History Session API — positive scenarios for both technical and viva modes, and the negative scenario for a missing 'mode' parameter — passed successfully.

4. API Testing

This section documents functional testing of the PrepMate backend APIs performed using Postman.

History Messages API

Endpoint

```
GET /api/v1/history/messages/{session_id}?mode={technical|viva}
```

Purpose

Retrieves the full conversation history for a given interview session, returning each question asked by the AI, the user's answer, and the corresponding evaluation (score, strength, improvement, and recommendation). The mode parameter switches between technical interview history and viva (project defense) history.

Request

```
GET http://localhost:8000/api/v1/history/messages/test_tech_001?mode=technical
GET http://localhost:8000/api/v1/history/messages/test_viva_001?mode=viva
```

Expected Result

- 200 OK
- Valid JSON array returned
- Each entry contains user_message, ai_message, and evaluation fields
- Session history correctly retrieved for the given session_id and mode

Actual Result

Technical mode (mode=technical): The API returned the complete question-answer history for session test_tech_001, including evaluation objects with score, strength, improvement, and recommendation for each answered question. For unanswered questions, the evaluation object was present but contained empty values, which is the expected behavior.

Viva mode (mode=viva): The API returned the complete history for session test_viva_001, including project-related questions, user responses, and detailed AI-generated evaluations with scores and recommendations.

Status Code

```
200 OK
```

JSON Verification

- Response received in valid JSON format
- Required fields (user_message, ai_message, evaluation) present in every object
- No missing or malformed fields observed
- Nested evaluation object (score, strength, improvement, recommendation) correctly structured

Screenshots

Technical mode — request and initial response:

The screenshot shows the Postman interface with a GET request to `http://localhost:8000/api/v1/history/messages/test_tech_001?mode=technical`. The response is a JSON object with the following structure:

```
1 {
2   "user_message": "Hello, I am ready to start my technical interview
3   stimulation",
4   "ai_message": "Explain the difference between a process and a thread. When
5   would you choose to use multiple processes versus multiple threads in an
6   application?",
7   "evaluation": {
8     "score": "",
9     "strength": "",
10    "improvement": "",
11    "recommendation": ""
12  },
13 }
14 {
15   "user_message": "A database is used to store and organize structure data. A
16   relational database uses row and tables.",
17   "ai_message": "Next Question\nCan you explain
18   the concept of database normalization and why it is important in
19   relational database design?",
20   "evaluation": {
21     "score": "5/10",
22   }
```

Technical mode — evaluation object detail (score, strength, improvement, recommendation):

The screenshot shows the same GET request in Postman, but the response is expanded to show the evaluation object details:

```
34   "ai_message": "Next Question\nDescribe the
35   concept of 'Big O' notation and explain its significance in algorithm
36   analysis.",
37   "evaluation": {
38     "score": "8/10",
39     "strength": "The candidate was direct and honest about their lack of
40     knowledge.",
41     "improvement": "The candidate needs to demonstrate foundational
42     knowledge or at least attempt to articulate what they do know, even
43     if incomplete, rather than simply stating 'I don't know.'",
44     "recommendation": "A process is an independent execution environment
45     with its own dedicated memory space, resources, and context. It's a
46     self-contained program in execution. Threads, conversely, are units
47     of execution within a process. Multiple threads can exist within a
48     single process, sharing the process's memory space and resources,
49     but each thread maintains its own program counter, stack, and
50     registers. Processes provide isolation, while threads enable
51     concurrency within a process, allowing for more efficient resource
52     utilization and responsiveness."
53   }
```


Viva mode — request and initial response:

The screenshot shows the Postman interface with a GET request to `http://localhost:8000/api/v1/history/messages/test_viva_001?mode=viva`. The response is a 200 OK status with a 12 ms response time and a 12.35 KB body. The response body is a JSON object with the following structure:

```
1 {
2   "user_message": "Hello, I am ready to start our viva stimulation",
3   "ai_message": "Could you please provide a concise overview of your project, including its main objectives and the problem it aims to address?",
4   "evaluation": {
5     "score": "",
6     "strength": "",
7     "improvement": "",
8     "recommendation": ""
9   }
10 },
11 {
12   "user_message": "Sorry, I failed to do so",
13   "ai_message": "Please provide a concise overview of your project, including its core aims and the primary research question guiding your work.",
14   "evaluation": {
15     "score": "",
16     "strength": "",
17     "improvement": "",
18     "recommendation": ""
19   }
20 },
21 }
```

The right sidebar shows the 'API Testing for User ...' section with a list of questions and a 'Describe what you need. Press @ for context, / for Skills.' prompt.

Viva mode — full response with recommendation field (end of session history):

The screenshot shows the Postman interface with the same GET request. The response is a 200 OK status with a 12 ms response time and a 12.35 KB body. The response body is a JSON object with the following structure:

```
98 {
99   "user_message": "Hello, I am ready to start our viva stimulation",
100   "ai_message": "Could you please provide a concise overview of your project, including its main objectives and the problem it aims to address?",
101   "evaluation": {
102     "score": "",
103     "strength": "",
104     "improvement": "The candidate could elaborate slightly more on the specific technical challenges encountered during the implementation phase or the unique contributions of their research beyond general efficiency improvements.",
105     "recommendation": "Good morning. My project, 'Smart Waste Management System Using IoT,' tackles the significant inefficiencies of static waste collection in urban environments, which often leads to overflowing bins in some areas and underutilized bins in others, driving up operational costs and environmental impact. We are developing an intelligent system leveraging low-cost ultrasonic sensors for real-time waste level monitoring, transmitting this data via a robust LoRaWAN network to a cloud-based analytics platform. The innovation lies in our dynamic routing algorithm, which processes this real-time data to create optimized collection paths, aiming to achieve a verifiable 38% reduction in fuel consumption and operational expenses compared to traditional fixed schedules, alongside improved urban sanitation. Our research specifically quantifies the performance gains of this data-driven approach and its overall socio-economic and environmental benefits, demonstrating a practical application of IoT for sustainable urban infrastructure."
106   }
107 },
108 {
109   "user_message": "Sorry, I failed to do so",
110   "ai_message": "Please provide a concise overview of your project, including its core aims and the primary research question guiding your work.",
111   "evaluation": {
112     "score": "",
113     "strength": "",
114     "improvement": "",
115     "recommendation": ""
116   }
117 },
118 }
```

The right sidebar shows the 'API Testing for User ...' section with a list of questions and a 'Describe what you need. Press @ for context, / for Skills.' prompt.

Result

✓ PASS

Final Summary

API	Method	Status Code	Result
Technical Chat	POST	200 OK	PASS
Viva Chat	POST	200 OK	PASS
Technical Upload	POST	200 OK	PASS
Viva Upload	POST	200 OK	PASS
Chat History	GET	200 OK	PASS
History Sessions	GET	200 OK	PASS
History Messages	GET	200 OK	PASS

Overall Conclusion

API testing for PrepMate – AI-Powered Technical Interview and Project Viva Simulator was successfully completed using Postman. The backend APIs for Technical Chat, Viva Chat, Technical Upload, Viva Upload, Chat History, History Sessions, and History Messages were tested to verify endpoint functionality, HTTP status codes, JSON response structure, request validation, and overall backend behavior. The APIs consistently returned valid responses for the tested scenarios, while validation mechanisms correctly handled invalid requests where applicable. The successful test results demonstrate that the backend services are functioning reliably and are ready for seamless integration with the frontend application for demonstration and further development.